TRASE ALGORITHM - AUTOMATIC EVOLUTIONARY MORPHING OF ELECTRONIC DANCE MUSIC

Rene Wooller
Creative Industries
Queensland University of Technology

Andrew Brown
Creative Industries
Queensland University of Technology

ABSTRACT

The *TraSe* (Transform – Select) algorithm has been developed to investigate the morphing of electronic music through automatically applying a series of deterministic compositional transformations to the source, guided towards a target by similarity metrics. This is in contrast to other morphing techniques such as interpolation of parameters or probabilistic variation. *TraSe* allows control over stylistic elements of the music through user-defined weighting of numerous compositional transformations. The formal evaluation of *TraSe* was mostly qualitative and occurred through nine participants completing an online questionnaire. The music generated by *TraSe* was generally felt to be less coherent than a human composed benchmark but in some cases judged as more creative.

1. INTRODUCTION

As with other morphing algorithms (Mathews and Rosler, 1969; Polansky 1987), *TraSe* operates on two note sequences – *source* and *target*, and produces a *morph,* which can serve as a musical hybrid and/or transition between the two. This paper outlines some of the overall process, the transformations used and the results of a formal questionnaire.

TraSe is an evolutionary music algorithm (Biles 2008) in that the source is mutated into a pool of 'offspring' from which only one is selected for further mutation. The only other previously known significant evolutionary algorithm applied to note sequence morphing I have found is described by Horner and Goldberg (1991). However, this differed in the range of compositional transformations that were applied, the way the transformations were applied and lack any accompanying music (morph examples shall be provided on ACMC CD) or formal analysis. For those interested in other, non-evolutionary works within the area of note sequence morphing, an extensive review is available (Wooller 2006).

2. OVERVIEW OF TRANSFORMATION SELECTION PROCESS

TraSe takes two note sequences, source, S, and target, T, and produces morphed material, M. The morph, M is a list of note sequence loops called **frames**, the first and last of which is S and T respectively. If we let n be the length of the frame list, $M_1 = S$ and $M_n = T$. The frames of the middle $M_2, M_{...}, M_{n-1}$ constitute a sequential progression.

During playback, the morph index, Ω, determines which frame in the series is playing. Letting i be the index of any frame, ranging from $1 \leq i \leq n$, and noting that Ω is normalized between 0 and 1, M_i will hold the note sequence that is looped when $\Omega = i/n$. Logically, the morph will be **smooth** if each frame is somehow made similar to its neighbours. Smoothness is the quality of continuity, whereby each segment is perceived as being more similar to the points directly ahead and behind than the others which are further away.

The musical representation includes loop lengths, note onsets, durations, dynamics, scale degrees/passing notes, (rather than absolute pitch) scale, key and octave**.** There are two separate *TraSe* algorithms that operate in parallel – one for key and scale and another for note-level data (note onset, duration, dynamic and scale degree/passing note). I refer to the former as a *key/scale morph* and the latter as a *note morph*. Both the note morph and key/scale morph have a separate M which can be of different lengths. *TraSe* processes described in this paper are for the note morph.

During playback, the current frame of the note morph is overlayed with the key/scale morph to produce MIDI note output (pitch, duration, onset and dynamic). Having the key/scale and note morphs separate affords more specific control over the tonal elements of the morph.

If S and T are different lengths, each frame will have a length equal to the lowest common multiple. For example, if S was 6 beats and T was 4, the loops in each frame of M would be 12 beats long. Requiring the frames to be the same length is practical; a uniform length affords automated comparisons between sequences, due to their notes occupying an identical space. It also affords the combination of material.

Recall that i is an index to M. *TraSe* sequentially fills each frame of M, starting with $i = 2$, and finishing with $i = n$. Each frame, M_i, is created by passing the previous frame, M_{i-1}, through a chain of compositional transformation functions: let this be the list C. The functions in the chain C perform musical transformations that a composer might attempt, for example *harmonise* creates or removes harmonies at particular intervals, while *rate* speeds up or slows down the music by particular commonly used ratios, looping when necessary.

To generate the frame M_i, the first transformation, C_1, is passed the previous frame, M_{i-1}. The second transformation is passed the output of the first, and this continues until the last transformation. The last transformation is fed the output of the

second last transformation to create M_i. Letting m equal the length of C, or $m = |C|$, this means: $M_i = C_m(C_{m-1}(C_{...}(C_1(M_{i-1}))))$. *TraSe* stops generating frames when the current frame is judged as being similar enough to the target, T, by a variable for 'cut-off' that is specified by the user. Each frame is part of a sequence of frames that is the morph: $M_1, \ldots, M_i, \ldots, M_n$.
The evolutionary component of *TraSe* is in the fact that each transformation produces a range of different sequences from which only one is selected, through a comparison with the target sequence. The transformations are 'mutations' and the comparison to the target is a 'fitness function'. Allowing specific compositional transformations and dissimilarity measures to be designed and plugged into the algorithm enables elements of compositional style to be specified.

3. TRANSFORMING AND SELECTING

Each transformation has a set of possible parameter configurations. A pool of candidate note sequences are created using each possible parameter configuration. A single candidate is then selected, using a fitness function that compares the candidate note sequences with the target note sequence.

Recall that the number of transformations is m, that is $m = |C|$. Let j be an index into the list of transformations, C, ranging over $1 \leq j \leq m$. For C_j, there are a related set of parameter configurations, P_j. The parameter configurations within P_j are used by C_j to create transformed note sequences within a 'selection pool'; let it be O_j. The 0th item in each parameter set is "bypass", $P_{j,0} = bypass$, thus enabling the unmodified input to also be an item in the pool.

For example, the *rate* transformation multiplies the start-time of each note by a ratio from the set $\frac{1}{4}, \frac{1}{2}, \frac{2}{3}, \frac{3}{2}, 2$ and 4. If *rate* is the 1st transformation, that is, $C_1 = rate$, the corresponding parameter set is: $P_1 = \{1, \frac{1}{4}, \frac{1}{2}, \frac{2}{3}, \frac{3}{2}, 2, 4\}$. While *rate* has a single parameter, more complex transformations include particular combinations of multiple parameters. Having a fixed set of values may seem like an unnecessary limitation when dealing with parameters that are otherwise continuous; however, it also affords precise control over parameter values and thus musical style. As well as this, the parameter sets can be any size and requiring discrete specification encourages careful consideration of the musical ramifications of each parameter value.

Each transformation C_j has an input note sequence; let it be I_j. Let k be an index to parameters in the set P_j and the corresponding output in O_j, that is, $0 \leq k \leq |P_j|$. Thus, the creation of the kth note sequence in the pool for the jth transformation is: $O_{j,k} = C_j(I_j, P_{j,k})$. Continuing the *rate* example, where $j = 1$, creating the 1st note sequence in the selection pool ($k = 1$) would be: $O_{1,1} = C_1(I_1, P_{1,1}) = C_1(I_1, \frac{1}{4})$. In this example, the start time for each note would be multiplied by $\frac{1}{4}$ and the result looped four times to make it the same length as the input. The diagram below provides an example of the complete selection pool for this *rate* example:

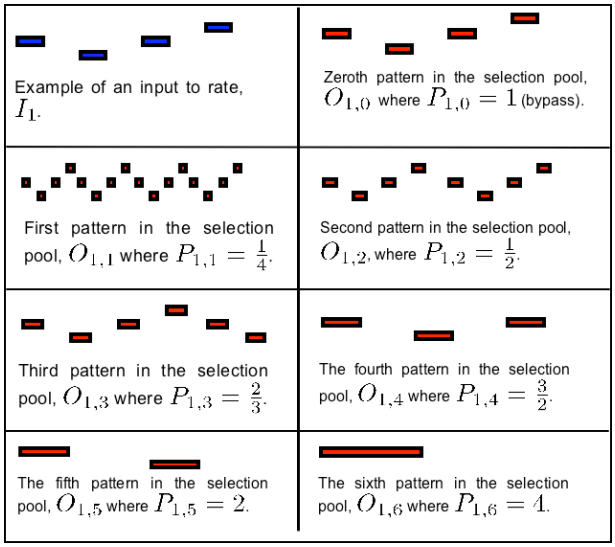


Figure 1 Each of the seven note sequences that would be in a selection pool for ***rate*** (j=1), given the original (top-left).

A fitness function determines which note sequence in the selection pool, O_j, is selected for output. Let the index of this selected note sequence in O_j be denoted by k^*. The input to the first transformation, C_1, is the previous frame (in the first cycle, this will be the source). Recalling that n is the number of frames and that $1 \leq i \leq n$, this means: $I_1 = M_{i-1}$. For transformations other than the first in the chain, the input is the selected output of the previous transformation in the chain, that is, for $2 \leq j \leq m$, $I_j = O_{j-1,k^*}$. Each frame other than the first is derived from the selected output of the last compositional transformation in the chain, that is, for $2 < i \leq n$, $C_m(C_{m-1}(C_{...}(C_1(M_{i-1})))) = O_{m,k^*} = M_i$. The first frame is the source music: $M_1 = S$.

4. SELECTION THROUGH DISSIMILARITY MEASURES

Each element in the selection pool is compared with T using a dissimilarity measure[1]. The selected note sequence, O_{j,k^*}, has a dissimilarity rating that is the closest to a particular target value. The process for determining the target value of dissimilarity with T is described further below. The application of user-defined weightings and "convergence" settings are also involved.

The dissimilarity measure for a particular transformation C_j,

[1] Discussing the measures in terms of 'dissimilarity' rather than similarity is an arbitrary choice, however, it is useful as 'dissimilarity' is analogous to 'distance', which is a natural method of comparing two values. The dissimilarity measure is not a distance metric in the formal mathematical sense, because triangle inequality is not necessarily upheld.

denoted by D_j, is designed to complement that transformation. For example, if $C_1 = rate$, D_1 calculates dissimilarities between the inter-onset envelopes, which are directly affected by the *rate* transformation. Each dissimilarity measure takes two note sequences and returns a value between 0 (similar) and 1 (dissimilar).

Each dissimilarity measurement between elements in the selection pool and the target is factored by a user controlled weight. The weights allow various parameter configurations to be unnaturally favoured which means the style of compositional transformations used throughout the morph can be influenced by the user.

Let w hold the user-defined weights. Recall that k is an index to the items in the parameter set P_j and the corresponding selection pool O_j. Let each item in the selection pool $O_{j,k}$ have a related weight, $w_{j,k}$. Let $R_{j,k}$ hold the result of the dissimilarity between $O_{j,k}$ and T, scaled by $w_{j,k}$. This means: $R_{j,k} = w_{j,k}\ D_j(O_{j,k}, T)$. Each measure in the set of weighted results R_j can be sorted from lowest to highest. Let the result of this sorting be $\overrightarrow{R}_j$. Thus, within this list, $k = 0$ refers to the smallest value (least dissimilar), and $k = |R_j|$ refers to the largest (most dissimilar). The selection pool can be arranged in the same order, so that $\overrightarrow{O}_{j,0}$ refers to the $O_{j,k}$ that minimises $w_{j,k}\ D_j(O_{j,k}, T)$.

5. SPEEDING THROUGH THE SCENIC ROUTE

If the goal of *TraSe* was to converge with T in as few frames as possible, then it would make sense to not use weights, and always choose $\overrightarrow{O}_{j,0}$, however, this is not the case. The aim instead is to find a musically interesting progression and this involves taking the "scenic" route. The **transform speed** is a user-defined parameter, s_j, which influences the extent to which the number of frames, n, is minimised, for each of the transformations in the chain. s_j is between 0, for the maximised or "slowest" setting, and 1, for the minimised or "fastest".

The inverse of s_j represents how many frames, n, can be expected using a single **perfect transformation**. A perfect transformation is a theoretical transformation that has the capacity to produce an infinite number of patterns in the selection pool, with an even spread of dissimilarity to T throughout. In a perfect transformation, when $s_j = \frac{1}{3}$, there will be 3 cycles before convergence. In practice, most transformations are imperfect and with some combinations of transformations and settings, convergence is never reached.

5.1. Target dissimilarity and transformation speed

From the user-defined transformation speed, s_j, a target value of dissimilarity, let it be t_j, needs to be established. This involves estimating how much the dissimilarity should be reduced by each step to ensure the specified number of steps, $1/s_j$, and multiplying it by the number of steps that have occurred already, i, to find the appropriate target, t_j, for this step. The estimate of the change in dissimilarity required for a single step is obtained by normalising s_j between the difference in least dissimilar, $\overrightarrow{R}_{j,0}$, which would be 0 with a **perfect** transformation, and the rating of the current note sequence that is the input, $R_{j,0}$ (recall that $P_{j,0} = bypass$). Considering that it would be unfeasible to pick a target that is below the lowest dissimilarity, $\overrightarrow{R}_{j,0}$ and recalling that i is the current frame, t_j is defined thus:

$$t_j = \max\left(\overrightarrow{R}_{j,0}, R_{j,0} - i\big(R_{j,0} - (1 - s_j)(R_{j,0} - \overrightarrow{R}_{j,0})\big)\right) \quad (1)$$

The equation above defines the target value of dissimilarity t_j. s_j is the user-defined 'speed' (0 is slowest). $\overrightarrow{R}_{j,0}$ is the lowest dissimilarity rating. $R_{j,0}$ is the dissimilarity rating of the unmodified input, that is, the previous frame.

This definition of t_j can be understood more intuitively thus: when the speed is set to slow, $s_j = 0$, it becomes $t_j = \max\left(\overrightarrow{R}_{j,0}, R_{j,0} - i\,\overrightarrow{R}_{j,0}\right)$. In the **perfect** case, where $\overrightarrow{R}_{j,0} = 0$ and $R_{j,0} = 1$, this becomes $t_j = \max(0, 1) = 1$. Note also that convergence does not occur in a single frame.

With this target (the most dissimilar), the algorithm will never converge, which is appropriate considering that as s_j approaches 0, $1/s_j$ (the number of steps) will approach ∞. If the speed is set to fast, $s_j = 1$ we obtain $t_j = \max\left(\overrightarrow{R}_{j,0}, R_{j,0} - iR_{j,0}\right)$, which, in the **perfect** case, becomes $t_j = \max(0, 1 - i)$.

In this case, for $1 \leq i \leq \infty$, the target similarity will be 0. This means that, in the **perfect** case, the algorithm would converge in a single frame, confirming that $1/s_j = n$.

5.2. Aiming for target dissimilarity while maintaining consistency

An appropriate k^* (the index of the selected sequence) will minimise the difference of measured dissimilarity, R_{j,k^*}, to the target dissimilarity, t_j, while at the same time minimising the dissimilarity of the selected sequence O_{j,k^*} to the source, S. This ensures that in situations where the candidates are rated equally close to t_j, the one that is the most similar to S will be selected, providing a greater sense of musical continuity. This will involve interpolating between the consistency with source and target for the currently desired dissimilarity. The difference between $R_{j,k}$ and the target dissimilarity t_j is: $|t_j - R_{j,k}|$. The distance from the source is $D(S, O_{j,k})$. Let v_j be a user-defined weight, called 'tracking VS consistency', which controls the balance between tracking t_j, when $v_j = 0$, and being consistent with S, when $v_j = 1$. This effectively controls the influence of $|t_j - R_{j,k}|$ or $D(S, O_{j,k})$. Usually, $v_j < 0.5$ so the difference to the target dissimilarity is the principal component. All together, k^* is

determined thus:

$$\min_{k^*} v_j \left| t_j - R_{j,k^*} \right| + (1 - v_j) D(S, O_{j,k^*}) \quad (2)$$

In the equation above, the index to the selected note sequence, k^*, is determined by minimising the difference of the dissimilarity of that note sequence with the target dissimilarity (first term above), while minimising the dissimilarity between that note sequence and the source (second term). The user can control the influence of each of these terms using v_j - the 'tracking VS consistency' variable.

6. COMPOSITIONAL TRANSFORMATIONS

The transformations that have been implemented are described here in order of their position in the transformation chain: *divide/merge*, *rate*, *phase*, *harmonise*, *scale pitch*, *inversion*, *octave*, *add/remove* and *key/scale*. All of these perform large scale transformations to the note sequences, except for *add/remove*, which deals with individual notes.

6.1. Divide/merge

The *divide/merge* transformation was envisaged as a technique for affecting the note density of the input without dramatically altering the music. *Divide/merge* has five parameter configurations, apart from 'bypass', and uses a 'Nearest Neighbour' dissimilarity measure to compare the transformed items in the selection pool with T. The five configurations are: merge forwards, merge backwards, split $\frac{1}{4}$, split $\frac{1}{2}$ and split $\frac{3}{4}$.

'Merge forwards' iterates through the note sequence, merging notes that overlap into one, retaining the pitch and dynamic of the earlier note. The end-time of the earlier note will increase to become the end-time of the later note and the later note will be removed. 'Merge backward' is similar, except that it proceeds from the last note to the first and the scale degree and dynamic are copied from the later note, deleting the earlier note.

'Split' finds the longest note and splits it in two. The three different parameter settings, $\frac{1}{4}$, $\frac{1}{2}$ and $\frac{3}{4}$ refer to the point at which the note is split. For example, splitting a whole note (four beats) with $\frac{1}{4}$ would turn it into one crotchet followed by a dotted minim. The scale degree and dynamic for both of these notes would be the same as the original.

To compare each note sequence generated by the difference parameter settings, *Divide/merge* uses the NN dissimilarity measure (below).

6.1.1. The Nearest Neighbour (NN) dissimilarity measure

The Nearest Neighbour (NN) dissimilarity measure is the most thorough of all dissimilarity measures within *TraSe*. For each note in the two note sequences being compared, the NN measure finds the distance of the closest (neighbouring) note in the opposite sequence. The final distance returned is the average of all such distances. This is currently an inefficient process at $O(n^3)$ although some ideas for optimisation have been explored which could bring it down to $O(n \log(n))$. This may be discussed in future research.

The process for NN is bidirectional, comparing the target sequence of notes, T, to the input, I, as well as I to T. These two calculations will be referred to as **backward** and **forward** respectively. The bi-directionality is necessary because the NN of one is not necessarily the NN of the other. For an example using note onset: if T has a note on beat 1 and 3 while I has a single note on beat 1.5, the note on beat 3 in T will be closest to the note in I and the note in I will be closest to the note on beat 1 in T.

Let A and B be two note sequences and let $av(A, B)$ be the average distance between each note in A and its NN B. Let i be an index to A that ranges over $1 < i \leq |A|$ ($|\ |$ is cardinality/length) and let j be an index to B that ranges over $1 < j \leq |B|$. Let $dist$ be a function that finds the distance between two different notes primarily in terms of pitch and note onset. The av function is thus:

$$av(A, B) = \frac{1}{|A|} \sum_{i=1}^{|A|} \min_{j=1}^{|B|} (dist(A_i, B_j)) \quad (3)$$

Equation three shows the average distance between each note in A and its NNs in B.

The *forward* calculation will be $av(I, T)$ and the *backward* calculation will be $av(T, I)$. Letting nnd be the NN dissimilarity measure, we have:

$$nnd(I, T) = \frac{1}{2} (av(I, T) + av(T, I)) \quad (4)$$

The note distance function for the NN measure is a combination of distance between note onsets within the loop and distance between "octavised" scale degrees. That is, the scale degree, d, plus the octave for that pitch, o, multiplied by the number of scale degree steps per octave, s: $d + os$. This scheme favours the use of source and target note sequences within the same octave. In future work it would be simple to overcome this by calculating the octave and scale degree distance separately and combining them afterward, weighted on a user-defined variable. Other measures that compare the similarity of scale degrees and passing notes could be applied. Inclusion of duration and dynamic and the Circle of Fifths (CF) distance, as explained in the previous chapter, would also be simple.

6.2. Rate

Rate multiplies the onset of each note by a ratio, removing notes that exceed the loop length, or looping the sequence as many times as needed to fill the loop. The ratios are based on patterns from Elctronic Dance Music genres: $\frac{1}{4}$, $\frac{1}{2}$, $\frac{2}{3}$, $\frac{3}{2}$, 2 and 4. For example, a straight beat transformed by $\frac{3}{2}$ or $\frac{2}{3}$ yields a common three against four rhythm. Transformations by $\frac{1}{2}$, and $\frac{1}{4}$ are notorious in breakdowns, while a rate change of 2 or 4 often increases the intensity of build-ups.

The dissimilarity measurement used for *rate* is the difference

in area between the onset envelopes, combined with the difference in area between the pitch envelopes (for an example, see Figure 2, below). The difference for inter-onset is normalised by the maximum possible difference area, which is the length of the loop squared. The difference for pitch is normalised by an arbitrary maximum of 30 multiplied by the loop length, which is equivalent to one note in each pattern, 30 semitones apart. If the difference happens to be greater than this, the maximum dissimilarity of 1 is returned. These measures are sufficient, but could be improved by normalising by the maximum (for pitch only) and magnifying the lower end of the spectrum through a logarithmic function.

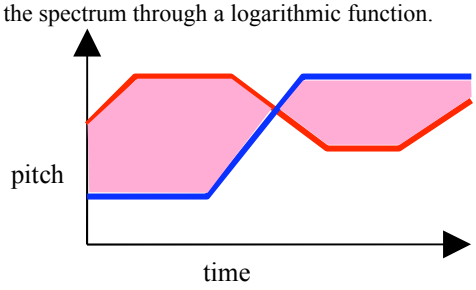


Figure 2 Two pitch envelopes and the difference in area between them. The difference in area between the onset and pitch envelopes are used as the dissimilarity measure for rate.

The result is probably similar to a NN measure. A comparison of the two measures is yet to be thoroughly investigated, however it is imagined that because *rate* preserves the contour of the envelope, the dissimilarity function that measures envelope differences is more relevant than the NN distances.

6.3. Phase

The *phase* transformation shifts the start time of the input loop by a certain amount, either ahead or behind. Onsets that exceed the loop boundary are wrapped around. The dissimilarity measure for *phase* is the NN dissimilarity measure, because of the high-level of accuracy in absolute difference between notes. The effect of *phase* is that the input pattern is "nudged" ahead or behind a certain amount, so as to maximise the absolute similarities between the patterns. This can have the musical effect of highlighting the similarities of notes relative to one and other (rather than relative to the metre) in the input and target patterns. The parameter space for *phase* is the series of quarter-intervals from -4 through to $+4$, that is, -4, -3.75, -3.5, $\cdots$, 3.5, 3.75, 4. This makes 32 different parameter settings, not including 0 or "bypass".

6.4. Harmonise

Harmonise works by either adding or removing parallel harmony at each of the tonal intervals from the 3rd through to the octave. This means 12 different parameter settings in all, excluding "bypass": remove 8^{ve}, 7th, 6th, 5th, 4th, 3rd; or add 8^{ve}, 7th, 6th, 5th, 4th or 3rd. While not all of these intervals are particularly common, they are included to provide flexibility.

To remove a harmonic interval, the note data is analysed to find the notes that occur on the same start time but with a different pitch – "harmonic clumps". Within each clump, notes that are the specified interval above the lowest pitch are removed. For the addition of harmonic intervals, monophonic notes are doubled with a parallel harmony at the specified interval.

The dissimilarity measure used by *harmonise* is a weighted combination of the difference in average harmonic interval and the difference in average clump size. The average harmonic interval is the sum of the average harmonic interval of each clump, normalized by the number of harmonic clumps (including clumps with a single note).

The weighting between average harmonic interval and average clump size is 0.8 and 0.2 respectively. This is because the most important effects rendered by *harmonise* are the changes in the harmonic interval, but clumps of various sizes can have the same average interval and so the average clump size is included so as to help differentiate between cases such as this.

6.5. Scale pitch

The *scale pitch* transformation expands or reduces pitch range, while maintaining contour. The pitch of each note is scaled by a certain ratio, relative to a Central Tonic (CT) pitch. There are fourteen ratio values, excluding "bypass", in the set of possible parameter configurations. These are all the intervals of $\frac{1}{7}$ from 0 to 2: $\frac{0}{7}$, $\frac{1}{7}$, $\cdots$, $\frac{13}{7}$, $\frac{14}{7}$. This was chosen so that it would be possible for an octave to be shifted to any of the other scale degrees above or below it.

The CT pitch is the tonic in the most central octave covered by the pitches of the input sequence. That is, the root of the octave in which the pitch centroid is found. Firstly, the average pitch of all the notes is calculated. This is then rounded down to the nearest tonic. For example, if in the key of C and the average pitch is B6, the tonic will be C6.

To scale the pitch, firstly the interval to the CT is found for each note pitch in the pattern by subtracting the CT from it. This interval is multiplied by the scaling ratio and the fundamental is added back to it. The scaled pitch is then adjusted to be in-scale or out-of-scale, to be consistent with the original value. For example, if it was a passing note (out of scale) to begin with, it will remain a passing note after the transformation.

An alternative *scale pitch* technique that avoids the need to bound the pitches is to determine the maximum possible ratio without exceeding the bounds, and weight this ratio with a parameter to determine the final scaling ratio that is used. This approach was trialled, however, the current approach was favoured as it was more predictable and the exceeding of bounds was rarely a problem.

The dissimilarity measure used for *scale pitch* is the difference in average interval from the CT, $avc(I)$. Recalling that I is a sequence of note pitches, n is the length of I, i is an index to I with the range $0 \leq i < n$ and c is the central tonic pitch, we have:

$$avc(I) = \frac{1}{n} \sum_{i} |I_i - c| \tag{5}$$

Equation five shows the average interval from the central tonic. I is the sequence of note pitches, n is the number of note pitches in I (the cardinal). i is an index to I and c is the central tonic.

This measure is used because the average interval from the central tonic is directly affected by the *scale pitch* function. One improvement to this measure could be to store and use the CT of the original sequence, rather than recomputing it for each of the scaled sequences. This is because in some situations the CT itself may be shifted, resulting in overly dramatic scaling ratios.

6.6. Inversion

Inversion operates in a similar way to "chord inversion", but inverts the pitches of the input sequence, I, rather than a chord. The degree of inversion is controlled by a parameter, let it be p, that ranges between $-1 \leq p \leq 1$. The number of octaves, q, by which to shift the pitches that are to be inverted must be greater than the octave range of the sequence, so that inverted notes do not clash with existing notes. Letting the number of steps per octave be denoted by s, we have $q = \frac{1}{s}(\max(I) - \min(I)) + 1$, rounded down[2].

The parameter p controls which pitches will be shifted and the direction. $|p|$ is the fraction of the range that will be shifted, and $sign(p)$ determines whether the percentage is taken from the top or bottom of the range and which direction it will shift. When $p > 0$ the selected pitches are taken from the bottom and shifted up and when $p < 0$ they are taken from the top and shifted down. For example, when $p = \frac{1}{2}$, all of the pitches that are below half the pitch range of the input pattern are shifted up by q octaves and when $p = -\frac{1}{2}$, all of the pitches that are above half the pitch range are shifted down by q octaves.

To select an inversion, the dissimilarity measure used is the difference in pitch envelopes, as described above in Rate (6.2). This was chosen because the inversion process has a substantial effect on the contour of the pitch envelope.

6.7. Octave

The *octave* transformation shifts the whole note sequence by a number of octaves, either: -3, -2, -1, 1, 2 or 3. The dissimilarity measure used for octave is simply the difference in average pitch.

6.8. Add/remove

The *add/remove* transformation guarantees that the output will be closer to the target, T, than the input, I. Each note in I is considered for removal and each note in T is considered for addition[3] and because of this, the parameter space for *add/remove* is not fixed like the other transformations. The NN dissimilarity measure (described above) is used to compare each result to T, so as to ensure accurate results. *Add/remove* is always the last transformation in the chain, enabling sequences that are not brought closer to T by any other transformation to eventually converge. Add/remove can be cycled a number of times specified by the user, feeding the output immediately back into the input. This is the 'add/remove cycles' parameter.

Add/remove operates in either 'polyphonic' or 'monophonic' mode. In monophonic mode, if a note is added at an onset that is already occupied, the existing note is replaced by the new note. Specifically, the pitch of the old note is replaced by that of the new note whereas the values of duration and dynamic for the new note are derived through combination between the old and new notes. The weighting of the combination is determined by the user. This feature was added after I had trialled source and target examples with very different durations and dynamics. It was found to add a degree of smoothness to the transition.

In polyphonic mode, the new note is overlaid, regardless of whether a note exists on the same onset. If a note exists on the same pitch (and onset), the attributes of the existing note are merged with the new note, using the same weighted combination parameter as monophonic mode.

Monophonic mode has greater transformational impact than polyphonic mode because, through replacement, an addition and removal can occur in a single step. Musically, especially in contexts that are mostly monophonic, the harmonies generated by polyphonic *add/remove* can sound unrealistic. A possible improvement could be for polyphonic mode to perform complete vertical replacements. That is, add or remove vertical note groups rather than single notes.

7. FORMAL QUALITATIVE EVALUATION: WEB QUESTIONNAIRE

A web questionnaire was developed as an empirical qualitative musicological investigation into morphing between MEM loops of various styles. The two primary aims were:

1) Collect knowledge that can be used to improve future morphing algorithms.
2) Establish if *LEMorpheus* is likely to be successfully applied to real-world contexts.

These were broken down further into more specific objectives:

1) Discover elements of *LEMorpheus* generated morphs that are perceived to have a positive or negative impact, benchmarking against human composed morphs.
2) Obtain techniques for morphing.
3) Determine if there is a correlation between smoothness and effectiveness. Smoothness is defined as similarity between successive sections, whereas effectiveness is defined as the ability the music to affect the listener in a way that is perceived to be the intention of the composer.
4) Obtain opinions as to whether the morphing techniques would be applicable to games or live

[2] Adding 1 and rounding down is chosen over simply rounding up, as it includes the octaves, which would not normally be rounded up.

[3] Adding notes from the target is a kind of 'cheat'. Ultimately, *TraSe* aims to function without deriving material directly from the target.

electronic music performance.

The approach was to research, compose and recreate sets of MEM loops, create *LEMorpheus* morphs between them and pay for a professional music producer to compose and produce morphs manually.

A randomly generated generated benchmark was also considered, however, human composed morphs were eventually chosen as they provided much more realistic and challenging material than randomly generated morphs and the time constraints of the questionnaire allowed for only the most effective benchmark. As well as this, random generation is only another human compositional decision and, as such, may have provided a false sense of objectivity to the study. Ultimately, comparing against cleverly composed music provides the kind of insights which may eventually be used to make *LEMorpheus* compose in a similarly clever way.

The questionnaire was developed according to the objectives, widely publicised and completed by nine participants with musical backgrounds. Australian respondents were paid and feedback was collated and analysed.

7.1. Results of questionnaire

In terms of results, *LEMorpheus* morphs were characterised by progressive musical variations, while the human composed morphs utilised layering and larger structural changes. While the majority of respondents favoured the human composed music, there was a great deal of controversy in opinion, probably due to participants attending more or less to different elements in the music and differences in musical background. As a taste of the controversy, two very different responses are included here:

"Excellent - because of the complexity of the tracks, the highly skilful melding of the source and target was a joy to hear. It was a very deliberate transition in moods, and almost evoked a story in itself as the listener was transported from a relaxed environment with the potential for disaster directly into a schizophrenic sound-scape. Very worthwhile."

- Participant five (15 years of avid listening to popular and alternative music), responding to the *LEMorpheus* morph for the second example.

"Blechh!! Intense negative. Not effective, an apparently random substitution of elements."

- Participant four (40 years of teaching electronic, art and popular music) responding to the *LEMorpheus* morph for the second example.

A number of suggestions for additional morphing techniques were gathered, including the blending of similar elements, removal of dissimilar elements, trading of loops (as with 'beat juggling'), incremental substitution of phrases, gauging of source and target dissimilarity to determine whether to attempt a continuous blend or switch directly, generation of a separate bridge section or breakdown and synchronisation of rhythm and evolution of other elements.

There was found to be very little correlation between smoothness and effectiveness. For example, many human composed morphs were found to be coherent but not smooth, while many *LEMorpheus* morphs were smooth but not coherent. Some terminological ambiguity with 'smoothness' was apparent in some responses, however, there was sufficient additional explanation for this not to impact significantly on the validity of the data.

Overall, *LEMorpheus* was rated, near-unanimously, as being applicable to both computer games and live electronic music contexts. The instances of negative results were somewhat balanced by similar reactions to the human composed morphs, although the balance was in favour of the human composed morphs. This was mostly due to the more deliberate and granular nature of the changes composed by the human, rather than the forced "smooth" transitions generated by *LEMorpheus*. The data from the questionnaire can be readily obtained through correspondence with the authors.

8. CONCLUSION

Most aspects of the evolutionary note-sequence morphing algorithm *TraSe* have been explained, along with an overview of the formal evaluation process. The core process involves iteratively feeding the source into a chain of compositional transformations and selecting from a pool of transformed candidates after each transformation. The result after each iteration of the chain is used as a single frame in a list that constitutes the morph. The method of selection for each transformation is to compare each candidate to the target to derive a measure of dissimilarity. The dissimilarity value for each candidate is weighted by user defined parameters and the candidate which is closest to a target level of dissimilarity is selected. The target level of dissimilarity is determined from the user-defined number of steps it would take to reach the target given an ideal transformation. The level of dissimilarity of the mutated note sequence to the source is also minimised where possible.

The following compositional transformations have been explained: divide/merge, rate, phase, harmonise, scale pitch, inversion, octave and add/remove. Divide/merge splits notes into separate notes while maintaining pitch, or merges notes with the same pitch. Rate transforms the onsets within the loop by certain ratios that are common to electronic dance music. Phase shifts onsets ahead or behind by quarter beat intervals. Harmonise either removes or adds harmonic intervals. Scale pitch either stretches or shrinks the pitch. Inversion shifts pitches at the upper or lower bounds to the lower or upper octaves respectively. Octave shifts the octave up or down. Add/remove individually adds or removes notes, ensuring eventual convergence with the target.

In the formal web-questionnaire evaluation, human composed source and target material was used, human adjustment of *TraSe* parameters allowed and the evaluation involved empirical qualitative feedback from a group of nine people with musical listening skills. Particular morphs were judged by some participants to be extremely novel and musically innovative. As well as this, the music generated by *TraSe* was considered overall to be applicable to real-world applications

such as computer games and electronic dance music. Despite this, the human composed morphs that were used as a benchmark were regarded as more appealing overall. Participants often differed in opinion because of different foci of attention at particular points in time and differing musical expectations based on their backgrounds.

Researching evolutionary approaches to compositional morphing of MEM has only just begun and there are many possibilities for future work that would yield substantial gain. This includes note thinning, to avoid muddiness and clashes; note clustering, to allow musical phrasing; new transformations, to extend the range of compositional capabilities; automated layering, to allow higher level structural features such as break-downs; and automatic adjustment of parameters, which would uncover less obvious – but workable – settings and reduce the time spent by the user. Load tests of the algorithm involving randomly generated material were performed and the results of these test may also be discussed in the future. As well as these, some ideas for optimisation of the algorithm may be implemented, which could allow truer realtime operation.

9. BIBLIOGRAPHY

Biles, A. 2008. “Evolutionary Music Bibliography”. accessed on: 15th of February, 2008. site: www.it.rit.edu/~jab/EvoMusic/EvoMusBib.html

Horner, A. and Goldberg, D. E. 1991. “Genetic Algorithms and Computer Assisted Composition”. *Proceedings of the International Computer Music Conference*. pp. 479-482.

Mathews, M. V. and Rosler, L. 1969. "Graphical Language for the Scores of Computer-generated Sounds". in *Music by Computers*, H. V. Foerster and J. W. Beauchamp. New York, John Wiley and Sons, Inc.**:** pp. 84-114.

Polansky, L. 1987. "Distance Music I-VI for any number of programmer/performers and live, programmable computer music systems". *Perspectives of New Music*. **25** (1)**:** pp. 537-544.

Wooller, R. 2006. "Review of compositional morphing: works, techniques, applications and possibilities". *Australasian Computer Music Conference ACMC2006:Trans*. Adelaide, ACMA.